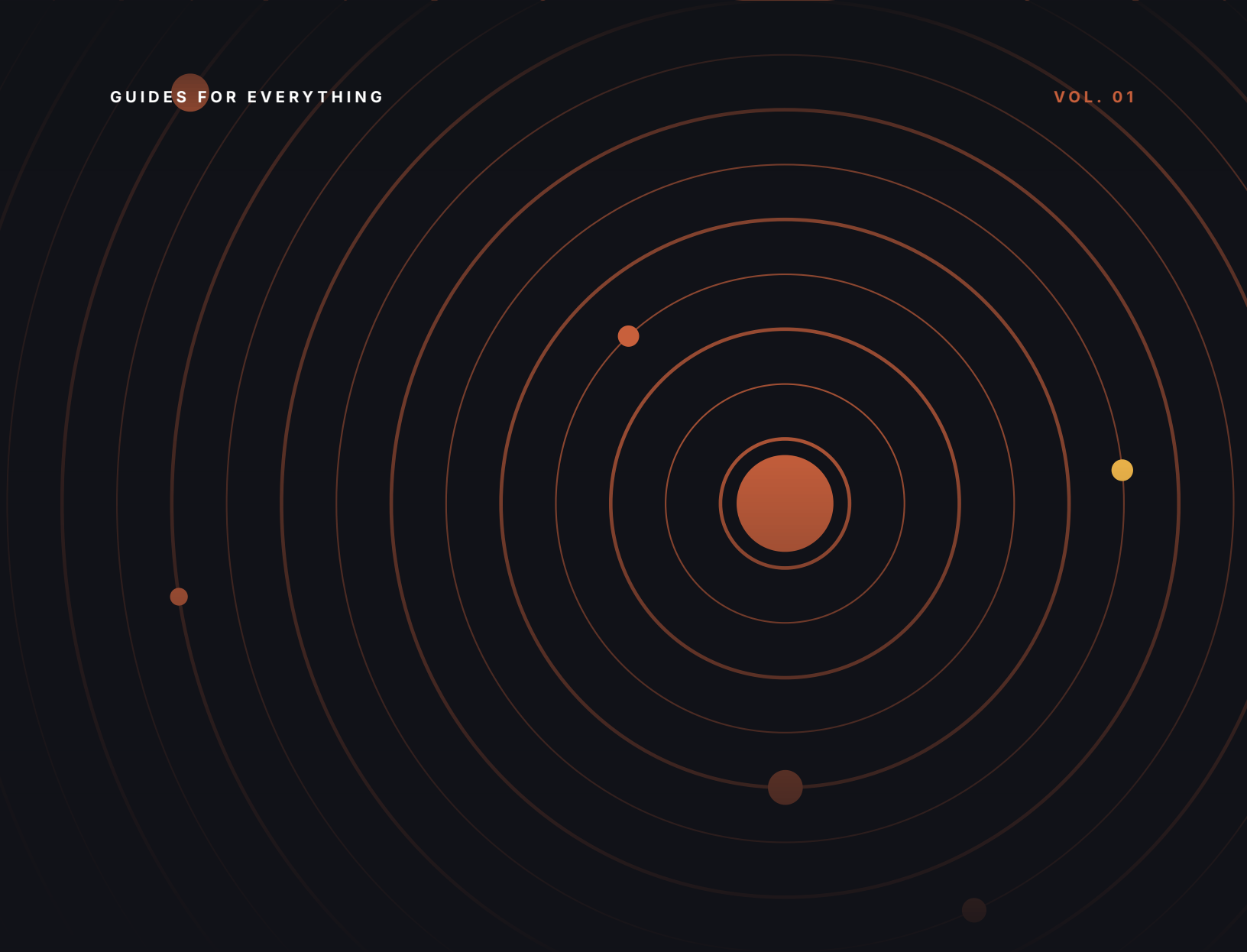


A BEGINNER'S MANUAL

# Claude Code

Your first AI coding partner, from installation to everyday mastery

For complete beginners • October 2026 edition • Print edition · US Letter

GUIDES FOR EVERYTHING

# Claude Code

Your first AI coding partner, from installation to everyday mastery

Claude Code is an AI assistant from Anthropic that can read your project, write and edit files, run commands, and explain everything it does in plain English. You talk to it the way you'd talk to a helpful colleague, and it does the work.

This manual assumes you have **never used a terminal before**. Each chapter builds on the one before it. By the end you'll have installed Claude Code, built and changed real files with it, learned to control what it is allowed to do, and set it up to remember how you like to work.

### HOW TO READ THIS MANUAL

- **Tip** — a shortcut or good habit.
- **Note** — useful background.
- **Warning** — easy to get wrong.
- **Danger** — can lose data, money, or access.
- `$ command` — text you type, exactly as shown.
- Settings › Privacy — a menu path to click.

New words appear in **bold** the first time and are collected in the Glossary at the back.

Facts, prices, and menu paths were checked against official documentation in October 2026. Software changes quickly — when something on screen doesn't match, trust the product's current docs.

# Contents


01 **Welcome to Claude Code** 5

1.1 What is an “agent”? 5

1.2 What can you do with it? 6

1.3 Where Claude Code runs 6

1.4 What you need 7

02 **Terminal Basics in Ten Minutes** 8

2.1 Opening a terminal 8

2.2 Folders and paths 9

2.3 The five commands you need 9

2.4 Copying and pasting in a terminal 10

03 **Install and Sign In** 11

3.1 Install with the native installer 11

3.2 Choose a plan 13

3.3 Log in for the first time 13

3.4 Health check and updates 14

04 **Your First Session** 15

4.1 The Claude Code screen 15

4.2 Build a small website 16

4.3 Ask questions about existing code 16

4.4 Approving, interrupting, and undoing 17

4.5 Ending and resuming 17

05 **Talking to Claude Effectively** 18

5.1 Five habits of great prompts 18

5.2 Plan mode: look before you leap 19

5.3 Share what you see 19

5.4 Ask a side question 19

5.5 Keep conversations focused 20

06 **Permissions and Safety** 21

6.1 Permission modes 21

6.2 Allow, ask, and deny rules 22
6.3 Where settings live 23
6.4 Safety nets 23

07 **Memory: Teaching Claude Your Project** 24
7.1 CLAUDE.md files 24
7.2 Create one with /init 25
7.3 What to put in it 25
7.4 Auto memory 26

08 **Everyday Commands and Shortcuts** 27
8.1 Slash commands 27
8.2 Keyboard shortcuts 28
8.3 Starting Claude Code from the terminal 29

09 **Extending Claude Code** 30
9.1 MCP: connect to other apps 30
9.2 Skills: your own slash commands 31
9.3 Subagents: specialist helpers 32
9.4 Hooks: automatic actions 32
9.5 Plugins: install bundles 33

10 **Beyond the Terminal** 34
10.1 The desktop app 34
10.2 VS Code and Cursor 34
10.3 Claude Code on the web and phone 34
10.4 Scripts and automation 35

11 **Costs and Limits** 36
11.1 How usage works 36
11.2 Make your usage go further 36

12 **Troubleshooting** 37

**Quick Reference** 38

**Glossary** 39

CHAPTER 1

# Welcome to Claude Code

Claude Code is an **AI coding agent**. That means it can do more than answer questions: it can open the files in your project, change them, run programs, check its own work, and keep going until a task is done. You stay in charge the whole time.

IN THIS CHAPTER

- What Claude Code is and what it's good for
- The different places you can use it
- What you need before you start

## 1.1 What is an "agent"?

You have probably used a **chatbot** such as the Claude app or ChatGPT. You type a question and it types back an answer. If it suggests code, *you* copy it, paste it into a file, and run it.

An **agent** closes that loop. When you ask Claude Code to "add a contact form to my website," it will:

1. Look through your project to understand how it's built.
2. Decide which files need to change.
3. Edit those files (asking your permission first, unless you've told it not to).
4. Run commands, for example starting your website, to check that the change works.
5. Tell you what it did, in plain language.

This cycle of *look → plan → act → check* is called the **agentic loop**. It's what makes Claude Code feel less like a search engine and more like a teammate.

## 1.2 What can you do with it?

You don't need to be a programmer to get value from Claude Code. People use it to:

- **Build things from a description.** "Make me a one-page website for my bakery with our hours and a menu."
- **Understand unfamiliar code.** "Explain what this project does, as if I'm new to coding."
- **Fix problems.** "When I run the app I get this error. Find out why and fix it."
- **Automate chores.** "Rename all these photos by the date they were taken."
- **Work with Git and GitHub.** "Commit my changes with a clear message and open a pull request."
- **Write documents and data files**, not just code: reports, spreadsheets, scripts, and more.

> **TIP**
>
> You can ask Claude Code about itself. Questions like "what can Claude Code do?" or "how do I add an MCP server?" work, because it has its own documentation built in.

## 1.3 Where Claude Code runs

Claude Code is one product with several "front doors." They share the same brain and the same settings, so what you learn in this manual works everywhere.

| SURFACE | WHAT IT LOOKS LIKE | BEST FOR |
|---|---|---|
| **Terminal (CLI)** | A text window where you type | The full feature set; the focus of this manual |
| **Desktop app** | The Claude app's **Code** tab | People who prefer buttons and windows over typing commands |
| **VS Code / Cursor extension** | A chat panel inside your code editor | Seeing edits side-by-side with your files |
| **JetBrains plugin** | Runs inside IntelliJ, PyCharm, etc. | Developers already using those editors |
| **Web** | claude.ai/code | Running tasks in the cloud, even from a phone |
| **Slack, GitHub Actions** | Mention Claude in a chat or pull request | Teams and automation |

**CLI** stands for *command-line interface*: a program you control by typing text commands instead of clicking.

## 1.4 What you need

- [ ] A computer running **macOS 13+**, **Windows 10 (1809+)**, or a recent **Linux** (Ubuntu 20.04+, Debian 10+), with at least 4 GB of RAM
- [ ] An internet connection
- [ ] A **paid Claude account**: Pro, Max, Team, or Enterprise, *or* an Anthropic Console account that pays per use
- [ ] A folder to work in (it can be empty, so you don't need an existing project)

> **NOTE**
>
> The free Claude plan does **not** include Claude Code. The cheapest way in is the Pro plan. See the pricing table in Chapter 3.

CHAPTER 2

# Terminal Basics in Ten Minutes

The terminal looks intimidating, but you only need a handful of ideas to use Claude Code comfortably. Think of it as a very literal text message conversation with your computer.

IN THIS CHAPTER

- How to open a terminal on Mac, Windows, and Linux
- Folders, paths, and the "current directory"
- The five commands you'll actually use

## 2.1 Opening a terminal

OPEN YOUR TERMINAL

1. **On a Mac.**
   Press `Cmd` + `Space`, type `Terminal`, and press `Enter`.
2. **On Windows.**
   Press the `Windows` key, type `PowerShell`, and open **Windows PowerShell** (or **Terminal**, which opens PowerShell by default).
3. **On Linux.**
   Press `Ctrl` + `Alt` + `T` on most systems, or search your apps for "Terminal."

You'll see a line ending in something like `%`, `$`, or `>`. That's the **prompt**: the terminal waiting for you to type. When this manual shows a command, type it after the prompt and press `Enter`.

> **TIP**
> On Windows, the prompt starts with `PS C:\` when you're in **PowerShell** and just `C:\` when you're in the older **Command Prompt (CMD)**. Claude Code works in both, but the install commands differ.

## 2.2 Folders and paths

Everything on your computer lives in **folders** (also called **directories**). A **path** is the address of a folder or file, like `/Users/sam/projects/bakery` on a Mac or `C:\Users\Sam\projects\bakery` on Windows.

The terminal is always "standing in" one folder, called the **current directory** or **working directory**. Claude Code works on the files in whatever folder you start it from. That's how it knows which project you mean.

The `~` symbol is shorthand for your **home folder** (for example `/Users/sam`).

## 2.3 The five commands you need

| COMMAND | WHAT IT DOES | EXAMPLE |
|---|---|---|
| `pwd` | Show which folder you're in ("print working directory") | `pwd` |
| `ls` | List the files here (`dir` in CMD) | `ls` |
| `cd` | Change into a folder | `cd projects/bakery` |
| `cd ..` | Go up one folder | `cd ..` |
| `mkdir` | Make a new folder | `mkdir bakery` |

Here's how to make a practice folder and step inside it:

TERMINAL

```
cd ~
mkdir my-first-project
cd my-first-project
pwd
```

> **TIP**
> Press Tab while typing a folder name and the terminal finishes it for you. Press the ↑ arrow to bring back the last command you typed.

## 2.4 Copying and pasting in a terminal

- **Mac:** Cmd + C and Cmd + V, as usual.
- **Windows Terminal / PowerShell:** Ctrl + V pastes; right-click also pastes.
- **Linux:** usually Ctrl + Shift + V.

In a terminal, Ctrl + C usually means **stop the running program**, not copy.

CHAPTER 3

# Install and Sign In

Installation takes about two minutes. Anthropic's recommended method is the **native installer**, which also keeps Claude Code up to date automatically.

IN THIS CHAPTER

- Installing on macOS, Linux, and Windows
- Choosing a plan and logging in
- Checking that everything works

## 3.1 Install with the native installer

INSTALL CLAUDE CODE

1. **Open a terminal.**
   See Chapter 2 if you're not sure how.

2. **Paste the command for your system and press Enter.**
   macOS, Linux, or WSL:

   TERMINAL

   ```
   curl -fsSL https://claude.ai/install.sh | bash
   ```

   Windows PowerShell:

   POWERSHELL

   ```
   irm https://claude.ai/install.ps1 | iex
   ```

   Windows Command Prompt (CMD):

COMMAND PROMPT

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del
install.cmd
```

3 **Wait.**
The installer shows no progress bar while it downloads. That's normal.

4 **Open a *new* terminal window**
so it picks up the new program.

5 **Check the install.**
Type `claude --version`. You should see a version number followed by `(Claude Code)`.

> ⚠ **"COMMAND NOT FOUND"?**
> If your terminal says `claude` isn't found or isn't recognized, the install folder isn't on your **PATH** yet. (The PATH is the list of folders your terminal searches for programs.) Close every terminal window and open a fresh one first. If it still fails, run the installer again and read its last few lines. It prints the exact fix.

## Other ways to install

| METHOD | COMMAND | UPDATES |
|---|---|---|
| **Homebrew (Mac)** | `brew install --cask claude-code` | Manual: `brew upgrade claude-code` |
| **WinGet (Windows)** | `winget install Anthropic.ClaudeCode` | Manual: `winget upgrade Anthropic.ClaudeCode` |
| **npm (needs Node.js 22+)** | `npm install -g @anthropic-ai/claude-code` | Manual |
| **apt / dnf / apk (Linux)** | See the setup page in the official docs | Through your system updates |

> ⚠ **DANGER**
> Never install with `sudo npm install -g`. Running npm as an administrator causes permission problems and is a security risk.

### Windows notes

- You do **not** need to run the installer as Administrator.
- Installing **Git for Windows** (from git-scm.com) is recommended. It gives Claude Code a Bash shell to run commands with. Without it, Claude Code uses PowerShell instead, which also works.
- If you already use **WSL** (Windows Subsystem for Linux), install from inside your WSL terminal using the macOS/Linux command.

## 3.2 Choose a plan

Claude Code needs a paid account. Prices below are from claude.com/pricing in October 2026:

| PLAN | PRICE | WHO IT'S FOR |
|---|---|---|
| **Free** | $0 | Chat only, **no Claude Code** |
| **Pro** | $20/month ($17/month billed yearly) | Learning and light daily use |
| **Max 5x** | $100/month | Heavy daily use: 5× Pro's usage |
| **Max 20x** | $200/month | All-day use: 20× Pro's usage |
| **Team** | $25/seat/month Standard, $125 Premium (less yearly) | Small teams, 2–150 people |
| **Enterprise** | Custom | Large organizations |
| **Console (API)** | Pay per use | Developers who prefer metered billing |

> **TIP**
> Start with **Pro**. If you regularly hit your usage limit, upgrade to Max later. Inside Claude Code, `/usage` shows how much of your limit you've used.

## 3.3 Log in for the first time

SIGN IN

1. **Go to your project folder.**
   For example: `cd ~/my-first-project`.
2. **Start Claude Code.**
   Type `claude` and press `Enter`.
3. **Pick a theme.**
   Use the arrow keys to choose light or dark.

4. **Choose how to log in.**
   Pick your Claude subscription (Pro/Max/Team/Enterprise) or Anthropic Console.
5. **Approve in your browser.**
   A browser window opens; click **Authorize**. Return to the terminal.
6. **Trust the folder.**
   Claude Code asks whether you trust the files in this folder. For your own projects, choose **Yes**.

You only log in once; Claude Code remembers your credentials. To switch accounts later, type `/login` inside Claude Code.

## 3.4 Health check and updates

- `claude doctor` checks your installation and settings and suggests fixes. Run it whenever something feels off.
- Native installs **update themselves** in the background. To update right now: `claude update`.
- Prefer slower, battle-tested releases? Type `/config` and set **Auto-update channel** to **stable** (about a week behind, skipping releases with major bugs).

CHAPTER 4

# Your First Session

Let's build something. In this chapter you'll start Claude Code, ask it questions, and have it create and change real files.

IN THIS CHAPTER

- What the Claude Code screen shows you
- Asking questions and requesting changes
- Approving actions, interrupting, and undoing

## 4.1 The Claude Code screen

When you run `claude`, you'll see:

- A **welcome panel** showing the version, the AI model in use, and your current folder.
- A **prompt box** where you type, marked with `>`.
- A **status bar** underneath showing the current *permission mode* (more on this in Chapter 6).

Type your request in plain English and press `Enter`. There's no special syntax to learn.

> **TIP**
>
> To write several lines before sending, press `Shift` + `Enter` for a new line (or type `\` then `Enter`, which works in every terminal).

## 4.2 Build a small website

YOUR FIRST PROJECT

1. **Start in an empty folder.**

   TERMINAL

   ```
   mkdir ~/bakery-site
   cd ~/bakery-site
   claude
   ```

2. **Describe what you want.**

   PROMPT

   > Create a simple one-page website for a bakery called "Rise & Shine".
   > Include our hours (Tue–Sun, 7am–3pm), a short menu with prices,
   > and a contact section. Keep it to plain HTML and CSS.

3. **Watch it plan and work.**
   Claude explains what it's about to do, then creates files such as `index.html` and `styles.css`.

4. **Approve when asked.**
   If Claude asks permission to create a file, choose **Yes**. (Chapter 6 explains how to approve whole categories at once.)

5. **Open the result.**
   Ask: `open the website in my browser`. Or open `index.html` from your file manager.

6. **Change it.**
   Try: `make the colors warmer and add a photo placeholder at the top`.

That's the whole rhythm: **describe, review, refine.**

## 4.3 Ask questions about existing code

Claude Code shines when you point it at a project someone else wrote. Start it inside the project folder and try:

PROMPT

> what does this project do?

PROMPT

explain the folder structure like I'm new to coding

PROMPT

where is the main entry point, and what happens when the app starts?

You don't need to tell Claude which files to read. It looks around by itself. If you *want* to point at a file, type `@` and start typing its name; Claude Code autocompletes the path.

## 4.4 Approving, interrupting, and undoing

**Permission prompts.** Before Claude edits a file or runs a command it isn't sure about, it may show what it intends to do and offer choices like:

- **Yes**: allow this one action.
- **Yes, and don't ask again**: allow this kind of action from now on in this project.
- **No**: refuse, and optionally tell Claude what to do instead.

**Interrupting.** Press Esc at any time to stop Claude mid-task. It keeps what it has done so far, and you can redirect it: "Stop. Use blue instead of green."

**Undoing.** Claude Code saves a **checkpoint** before each change. Press Esc twice on an empty prompt (or type `/rewind`) to roll the conversation *and/or* your files back to an earlier point.

> ⚠ WARNING
>
> Checkpoints cover file edits Claude makes. They can't undo things outside your computer, like a sent email, a deleted cloud file, or a database change. For real safety, use Git (Chapter 6 and the GitHub manual in this series).

## 4.5 Ending and resuming

| TO... | DO THIS |
|---|---|
| **Leave Claude Code** | Type `/exit`, or press Ctrl + D twice |
| **Start fresh in the same window** | `/clear` |
| **Continue your last conversation** | `claude -c` (from the terminal) |
| **Pick an older conversation** | `claude -r`, or `/resume` inside Claude Code |
| **Name a session to find it later** | `/rename bakery-site` |

CHAPTER 5

# Talking to Claude Effectively

Claude Code is capable, but it isn't a mind reader. A little care in how you ask makes a huge difference in what you get back.

IN THIS CHAPTER

- The habits that produce great results
- Plan mode: think first, then build
- Sharing screenshots, errors, and files

## 5.1 Five habits of great prompts

1. **Be specific.** Instead of "fix the bug," say "users see a blank screen after entering a wrong password, so fix that."
2. **Give the goal, not just the task.** "I want customers to order ahead online" helps Claude make better choices than "add a form."
3. **Break big jobs into steps.** Number them, and Claude will work through them in order.
4. **Let Claude explore first.** "Read the code and explain how login works before changing anything."
5. **Say how you'll judge success.** "When you're done, run the tests and show me they pass."

PROMPT

1. Add a "Order ahead" page with a form for name, item, and pickup time.
2. Save orders to a file called orders.csv.
3. Add a link to the new page in the site's menu.
4. Open the site and confirm the form works before you finish.

## 5.2 Plan mode: look before you leap

For anything bigger than a small tweak, ask Claude to **plan first**. In **plan mode** Claude researches and writes a step-by-step plan but won't edit your files until you approve it.

- Press Shift + Tab until the status bar says `plan mode on`, **or**
- Start your message with `/plan`, for example: `/plan add online ordering`.

When the plan appears you can approve it, ask for changes ("No, keep planning, and don't use a database"), or press Ctrl + G to edit the plan yourself in a text editor.

> **TIP**
>
> Plan mode is the single best habit for beginners. Reading the plan teaches you how the project works, and it catches misunderstandings before any files change.

## 5.3 Share what you see

- **Error messages:** copy and paste them straight into the prompt. Include the whole thing.
- **Screenshots:** paste an image with Ctrl + V (Cmd + V in iTerm2, Alt + V on Windows). Claude can see it.
- **Files:** type `@` followed by part of a file name.
- **A command's output:** start a line with `!` to run a shell command yourself, for example `!npm test`. Claude sees the result and responds to it.

## 5.4 Ask a side question

Want to ask something without derailing the current task? Use `/btw`:

**PROMPT**

```
/btw what's the difference between HTML and CSS?
```

The answer appears without being added to the main conversation.

## 5.5 Keep conversations focused

Claude can hold a lot in mind at once, but that space, the **context window**, is not unlimited. Long, wandering sessions get slower and cost more of your usage.

- Use `/clear` when you switch to an unrelated task.
- Use `/compact` to summarize a long conversation and free up space while staying on the same task.
- Use `/context` to see how full the context window is.

CHAPTER 6

# Permissions and Safety

Claude Code can edit files and run commands on your computer, so it comes with a permission system: rules for what it may do without asking. Understanding it lets you work faster *and* safer.

IN THIS CHAPTER

- The six permission modes and how to switch
- Allow, ask, and deny rules
- Safety nets: checkpoints, Git, and the sandbox

## 6.1 Permission modes

A **permission mode** sets how much Claude can do before checking with you. Press Shift + Tab to cycle modes during a session. The status bar shows the active one.

| MODE | RUNS WITHOUT ASKING | GOOD FOR |
|---|---|---|
| **Manual (`default`)** | Reading files only | Sensitive work; learning what Claude does |
| **Accept edits (`acceptEdits`)** | Reading, editing files, and simple file commands like `mkdir`, `mv`, `cp` | Iterating on code you'll review afterwards |
| **Plan (`plan`)** | Reading and exploring; no edits until you approve a plan | Thinking before building |
| **Auto (`auto`)** | Almost everything, with a safety **classifier** checking each action in the background | Long tasks with few interruptions |
| **Don't ask (`dontAsk`)** | Only pre-approved tools; everything else is refused | Scripts and automation |
| **Bypass (`bypassPermissions`)** | Everything, no checks | Disposable containers and virtual machines **only** |

**Auto mode** is the starting mode for new terminal and VS Code sessions on recent versions of Claude Code. A second AI model (the **classifier**) reviews each risky action, such as running a script from the internet, force-pushing code, deploying, or deleting files that existed before the session, and blocks anything that goes beyond what you asked for.

> ☆ IMPORTANT
>
> You can tell auto mode about limits in plain English: "don't push anything until I've reviewed it." The classifier treats that as a rule for the rest of the conversation. For a hard guarantee, add a **deny rule** (below).

To start a session in a particular mode:

TERMINAL

```
claude --permission-mode plan
```

## 6.2 Allow, ask, and deny rules

Rules let you pre-approve safe actions and forbid dangerous ones. Type `/permissions` to see and edit them, or write them in a settings file.

- **Allow**: run without asking.
- **Ask**: always ask first, even in auto mode.
- **Deny**: never allowed, in any mode.

Deny rules win over ask rules, which win over allow rules.

.CLAUDE/SETTINGS.JSON

```
{
  "permissions": {
    "allow": [
      "Bash(npm run *)",
      "Bash(git commit *)"
    ],
    "ask": [
      "Bash(git push *)"
    ],
    "deny": [
      "Read(./.env)",
      "Bash(rm -rf *)"
    ]
  }
}
```

The `*` matches anything, so `Bash(npm run *)` covers `npm run build`, `npm run test`, and so on. Put the `*` *after* the sub-command: `Bash(git commit *)` is narrow, but `Bash(git *)` allows every Git command.

> **TIP**
> Tired of approving the same harmless commands? Type `/fewer-permission-prompts`. Claude scans your recent sessions and proposes a safe allow-list for read-only commands you use often.

## 6.3 Where settings live

Claude Code reads settings from several files. When the same setting appears in more than one place, the higher one in this list wins:

| PRIORITY | FILE | APPLIES TO |
|---|---|---|
| 1 (highest) | Managed settings from your organization | Everyone in your company |
| 2 | `claude --settings …` on the command line | This session |
| 3 | `.claude/settings.local.json` | You, in this project (not shared) |
| 4 | `.claude/settings.json` | Everyone working in this project |
| 5 | `~/.claude/settings.json` | You, in every project |

`~` means your home folder. A file inside `.claude/` lives in the project folder.

## 6.4 Safety nets

- **Checkpoints:** `/rewind` (or Esc Esc) restores earlier file versions.
- **Git:** commit your work before big changes so you can always get back. Ask Claude: "commit everything with a clear message before you start."
- **Protected files:** Claude never silently edits things like `.git`, `.bashrc`, or its own `.claude` settings. It always asks, except in bypass mode.
- **Sandbox:** type `/sandbox` (macOS, Linux, WSL2) to run Claude's shell commands in an isolated box with limited file and network access.

> **BYPASS MODE IS FOR THROWAWAY MACHINES**
> `--dangerously-skip-permissions` turns off every check. A bad instruction hidden in a web page or file (a **prompt injection**) could then make Claude do real damage. Only use it inside a container or virtual machine you're happy to wipe. The *Maximum Autonomy* guide in this series covers how to set that up.

CHAPTER 7

# Memory: Teaching Claude Your Project

Every session starts with a blank slate. Two features carry knowledge from one session to the next: **CLAUDE.md files**, which you write, and **auto memory**, which Claude writes.

IN THIS CHAPTER

- What a CLAUDE.md file is and where to put it
- Generating one with `/init`
- What belongs in it, with an example

## 7.1 CLAUDE.md files

A `CLAUDE.md` file is a plain-text note, written in **Markdown** (a simple way to format text with `#` headings and `-` bullets), that Claude reads at the start of every session. Use it for anything you'd otherwise re-explain each time: how to run the project, house style, "always do X" rules.

| WHERE | FILE | WHO IT'S FOR |
|---|---|---|
| **Your whole computer** | `~/.claude/CLAUDE.md` | Your personal preferences, every project |
| **A project (shared)** | `./CLAUDE.md` or `./.claude/CLAUDE.md` | Everyone on the project, checked into Git |
| **A project (just you)** | `./CLAUDE.local.md` | Your private notes; add it to `.gitignore` |

## 7.2 Create one with /init

Inside a project, type:

PROMPT

```
/init
```

Claude studies the project and writes a starter `CLAUDE.md` with build commands, test instructions, and conventions it finds. If one already exists, it suggests improvements instead of overwriting.

## 7.3 What to put in it

CLAUDE.MD

```
# Rise & Shine bakery website

## How to preview
- Open index.html in a browser. No build step.

## Style
- Plain HTML and CSS only. No frameworks.
- Colors: warm cream (#FFF6E9) and cinnamon (#B5562B).
- Write friendly, short copy. No jargon.

## Rules
- Keep every page working on phones.
- Ask before deleting any image.
```

Tips from Anthropic's guidance:

- **Keep it short:** aim for under about 200 lines. Long files get followed less reliably.
- **Be concrete:** "Use 2-space indentation" beats "format code nicely."
- **Add a line when Claude makes the same mistake twice.**
- **Import other files** with `@`, e.g. `See @README.md for an overview.`

> **NOTE**
>
> CLAUDE.md is *guidance*, not a lock. To truly block an action, use a deny rule (Chapter 6) or a hook (Chapter 9).

## 7.4 Auto memory

As you work, Claude can note your corrections and preferences ("Sam prefers short commit messages") in its own memory files, which load automatically next time. Type `/memory` to view or edit them, open your CLAUDE.md files, or turn auto memory on or off.

CHAPTER 8

# Everyday Commands and Shortcuts

You don't need to memorize these. Type `/` at any time to see a searchable menu of everything available. Here are the ones you'll use most.

## 8.1 Slash commands

| COMMAND | WHAT IT DOES |
|---|---|
| `/help` | Show help and available commands |
| `/init` | Create a `CLAUDE.md` for this project |
| `/clear` | Start a new, empty conversation |
| `/compact` | Summarize the conversation to free up space |
| `/context` | Show how full the context window is |
| `/model` | Switch AI model (e.g. `opus`, `sonnet`, `haiku`) |
| `/plan` | Enter plan mode, optionally with a task |
| `/permissions` | View and edit allow / ask / deny rules |
| `/rewind` | Roll back the conversation and/or files |
| `/resume` | Reopen an earlier conversation |
| `/diff` | Review all changes made so far |
| `/review` | Ask Claude to review your changes for bugs |
| `/memory` | Edit CLAUDE.md files and auto memory |
| `/mcp` | Manage connected MCP servers (Chapter 9) |
| `/usage` | See plan limits and usage ( `/cost` works too) |

| | |
|---|---|
| `/config` | Open settings: theme, model, editor mode... |
| `/doctor` | Diagnose and fix setup problems |
| `/add-dir` | Let Claude work in an extra folder |
| `/export` | Save the conversation as text |
| `/btw` | Ask a side question |
| `/exit` | Quit |

## 8.2 Keyboard shortcuts

| KEYS | ACTION |
|---|---|
| Esc | Stop Claude mid-task |
| Esc Esc | Clear your draft; on an empty prompt, open rewind |
| Shift + Tab | Cycle permission modes |
| Shift + Enter | New line without sending |
| ↑ / ↓ | Scroll through previous prompts |
| Ctrl + R | Search prompt history |
| Ctrl + C | Interrupt; on an empty prompt, press twice to exit |
| Ctrl + O | Show the detailed transcript of what Claude did |
| Ctrl + G | Edit your prompt (or a plan) in a text editor |
| Ctrl + V | Paste an image |
| Ctrl + B | Send a long-running command to the background |
| ? | Show the shortcut help panel (on an empty prompt) |

## 8.3 Starting Claude Code from the terminal

| COMMAND | WHAT IT DOES |
|---|---|
| `claude` | Start an interactive session |
| `claude "fix the build error"` | Start with a first request |
| `claude -c` | Continue the most recent conversation here |
| `claude -r` | Choose an older conversation to resume |
| `claude -p "summarize README.md"` | Answer once and exit (handy in scripts) |
| `claude --model opus` | Use a specific model this session |
| `claude --permission-mode plan` | Start in a specific permission mode |
| `claude -w new-feature` | Work in an isolated copy of the repo (a Git *worktree*) |
| `claude update` | Update now |
| `claude doctor` | Check installation health |

CHAPTER 9

# Extending Claude Code

Out of the box, Claude Code can work with files, run commands, and search the web. You can extend it in five ways. Each has its own job:

| FEATURE | IN ONE SENTENCE | LIVES IN |
|---|---|---|
| **MCP servers** | Connect Claude to outside tools: GitHub, Notion, databases, browsers | `claude mcp add …` |
| **Skills** | Save a reusable set of instructions as a `/command` | `.claude/skills/<name>/SKILL.md` |
| **Subagents** | Specialist helpers with their own instructions and tools | `.claude/agents/<name>.md` |
| **Hooks** | Run your own command automatically when something happens | `"hooks"` in settings.json |
| **Plugins** | Bundles of the above you can install in one step | `/plugin` |

## 9.1 MCP: connect to other apps

**MCP** (Model Context Protocol) is an open standard that lets AI tools plug into other software. An **MCP server** is a small connector that gives Claude new abilities, such as reading your Notion pages or creating GitHub issues. (This series has a whole manual on MCP.)

Most modern services offer a **remote** server: a web address. Add one like this:

TERMINAL

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
```

Then type `/mcp` inside Claude Code, select the server, and sign in through your browser.

Some servers run as a program on your own computer (a **local** or **stdio** server). Everything after the `--` is the command that starts it:

TERMINAL

```
claude mcp add --env API_KEY=your-key myserver -- npx -y @example/mcp-server
```

| COMMAND | WHAT IT DOES |
|---|---|
| `claude mcp list` | List servers and whether they're connected |
| `claude mcp get <name>` | Show details for one server |
| `claude mcp remove <name>` | Remove a server |
| `--scope user` | Make a server available in all your projects |
| `--scope project` | Save it in `.mcp.json` to share with your team |

> ⚠ **WARNING**
>
> Only install MCP servers you trust. A server can see what Claude sends it and can return text that tries to trick Claude. Prefer official servers published by the company whose product you're connecting.

## 9.2 Skills: your own slash commands

A **skill** is a folder containing a `SKILL.md` file of instructions. Claude uses it automatically when it fits the situation, or you run it by name.

~/.CLAUDE/SKILLS/WEEKLY-UPDATE/SKILL.MD

```
---
description: Writes a friendly weekly progress update from recent Git commits. Use when asked
for a status update or weekly summary.
---

Look at the Git commits from the last 7 days. Write a short update with:
- three bullet points of what got done
- one bullet of what's next
Use plain language a non-programmer would understand.
```

Now `/weekly-update` works in every project. Put the folder in a project's `.claude/skills/` instead to share it with your team.

## 9.3 Subagents: specialist helpers

A **subagent** is a helper Claude can hand work to. Each has its own instructions, its own allowed tools, and its own fresh context, so a big research job doesn't clutter your main conversation. The easiest way to make one is to ask:

PROMPT

Create a project subagent called "copy-editor" that checks my website text for spelling, tone, and reading level. It should only be able to read files.

Claude saves it as `.claude/agents/copy-editor.md`. Claude Code also ships with built-in subagents, such as **Explore** (fast, read-only searching) and **Plan**.

## 9.4 Hooks: automatic actions

A **hook** runs a command of yours at a specific moment, for example after every file edit or when Claude needs your attention. Unlike CLAUDE.md, hooks *always* run. This example shows a desktop notification on a Mac whenever Claude is waiting for you:

~/.CLAUDE/SETTINGS.JSON

```
{
  "hooks": {
    "Notification": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "osascript -e 'display notification \"Claude needs you\" with title
\"Claude Code\"'"
          }
        ]
      }
    ]
  }
}
```

Type `/hooks` to see what's configured. You can also just describe the hook you want and ask Claude to set it up.

## 9.5 Plugins: install bundles

A **plugin** packages skills, subagents, hooks, and MCP servers together. Type `/plugin` to browse and install from **marketplaces** (catalogs of plugins). Anthropic's official marketplace is built in.

CHAPTER 10

# Beyond the Terminal

Everything you've learned carries over to Claude Code's other surfaces.

## 10.1 The desktop app

Download the Claude app from claude.com/download, sign in, and open the **Code** tab. You get the same Claude Code with a graphical interface: a folder picker, a visual **diff** (a side-by-side view of what changed), a permission-mode selector next to the send button, and the ability to run several sessions in parallel.

## 10.2 VS Code and Cursor

In VS Code or Cursor, open the Extensions view (`Ctrl`/`Cmd` + `Shift` + `X`), search for **Claude Code**, and click **Install**. A Claude panel appears in the sidebar. You can review proposed changes as inline diffs and @-mention the lines you've selected.

## 10.3 Claude Code on the web and phone

At claude.ai/code, or in the **Code** tab of the Claude mobile app, you can start a **cloud session**: Claude works on a copy of your GitHub repository on Anthropic's computers, so it keeps going even when your laptop is closed. When it's done, it pushes a branch you can review. Cloud sessions are available on Pro, Max, Team, and eligible Enterprise seats, and need your GitHub account connected.

From the terminal, `claude --cloud "fix the login bug"` hands a task to the cloud, and `claude --teleport` pulls a cloud session back to your computer.

## 10.4 Scripts and automation

`claude -p` (“print mode”) answers once and exits, which makes Claude Code usable inside scripts:

TERMINAL

```
cat error.log | claude -p "explain this error in one paragraph"
```

For GitHub, type `/install-github-app` to let your team mention `@claude` in issues and pull requests to get work done automatically.

CHAPTER 11

# Costs and Limits

## 11.1 How usage works

On a **subscription** (Pro, Max, Team), Claude Code use is included up to your plan's limits, which reset over time. On a **Console** account you pay per **token** (a token is a chunk of text, roughly ¾ of a word). Longer conversations and bigger files use more tokens.

Type `/usage` to see your current usage, your limits, and a breakdown of what's using the most.

## 11.2 Make your usage go further

- `/clear` **between unrelated tasks.** Old conversation history is re-read with every message.
- `/compact` **during long tasks** to shrink the history.
- **Use a lighter model for simple jobs:** `/model haiku` or `/model sonnet` for routine work, and save the most powerful model for hard problems.
- **Be specific.** A precise request avoids wasted exploration.
- **Keep CLAUDE.md lean.** It's loaded into every session.

> **TIP**
>
> If you hit a limit mid-task, `/rate-limit-options` shows your choices: wait for the reset, add usage credits, or upgrade.

CHAPTER 12

# Troubleshooting

| PROBLEM | TRY THIS |
|---|---|
| `claude: command not found` | Open a new terminal window. Still failing? Re-run the installer and follow the PATH fix it prints, or run `claude doctor`. |
| `'irm' is not recognized` | You're in CMD, not PowerShell. Use the CMD install command, or open PowerShell. |
| `The token '&&' is not a valid statement separator` | You're in PowerShell. Use the PowerShell command instead. |
| **Login window doesn't open** | Copy the link Claude Code prints into your browser manually. |
| **Claude keeps asking permission for the same thing** | Choose **Yes, and don't ask again**, add an allow rule, or run `/fewer-permission-prompts`. |
| **Claude ignores an instruction** | Put it in CLAUDE.md, make it concrete, and keep the file short. For must-never-happen rules, use a deny rule. |
| **Responses getting slow or confused** | `/compact` or `/clear`, then restate the goal. |
| **Something is broken and you don't know why** | `/doctor` inside Claude Code. It diagnoses and offers fixes. |
| **MCP server shows "Failed to connect"** | `claude mcp get <name>` shows the error; re-authenticate in `/mcp`. |

> **NOTE**
>
> Stuck? Type `/bug` to report a problem to Anthropic, ask Claude itself ("why did that fail?"), or take the free Claude Code courses at academy.claude.com.

# Quick Reference

**Install (Mac/Linux):** `curl -fsSL https://claude.ai/install.sh | bash`
**Install (Windows PowerShell):** `irm https://claude.ai/install.ps1 | iex`
**Start:** `cd my-project` then `claude` · **Continue:** `claude -c` · **Resume:** `claude -r`

**Stop:** Esc · **Undo:** Esc Esc or `/rewind` · **Modes:** Shift + Tab · **New line:** Shift + Enter

**Plan first:** `/plan …` · **Fresh start:** `/clear` · **Shrink history:** `/compact` · **Project memory:** `/init`

**Connect an app:** `claude mcp add --transport http <name> <url>` then `/mcp` to sign in

**Health check:** `claude doctor` · **Update:** `claude update` · **Usage:** `/usage`

# Glossary

| | |
|---|---|
| **Agent** | An AI that can take actions, such as reading and editing files or running commands, in a loop until a task is finished, rather than only replying with text. |
| **Agentic loop** | The cycle an agent repeats: gather context, plan, act, check the result. |
| **Auto mode** | A permission mode where a separate safety classifier reviews Claude's actions instead of asking you about each one. |
| **Bypass permissions** | A permission mode (`--dangerously-skip-permissions`) that skips all checks. Meant only for isolated, disposable environments. |
| **Checkpoint** | An automatic snapshot taken before Claude edits files, which you can return to with `/rewind`. |
| **Classifier** | A second AI model that judges whether an action in auto mode matches what you asked for. |
| **CLAUDE.md** | A Markdown file of instructions Claude reads at the start of every session. |
| **CLI** | Command-line interface: a program you control by typing commands in a terminal. |
| **Context window** | The amount of conversation and file content Claude can consider at once. |
| **Current directory** | The folder your terminal is "in." Claude Code works on the files in this folder. Also called the working directory. |
| **Diff** | A view showing exactly which lines of a file were added, removed, or changed. |
| **Git** | A tool that records the history of your files so you can review changes and go back in time. |
| **Hook** | A command you configure to run automatically at a set moment, such as after every file edit. |

| | |
|---|---|
| **Markdown** | A simple text format that uses symbols like `#` and `-` for headings and lists. |
| **MCP** | Model Context Protocol, an open standard for connecting AI tools to other apps and data. |
| **MCP server** | A connector that gives an AI new tools, such as access to Notion, GitHub, or a database. |
| **Model** | The specific AI "brain" doing the work, e.g. Opus (most capable), Sonnet (balanced), Haiku (fastest). |
| **PATH** | The list of folders your terminal searches when you type a program's name. |
| **Permission mode** | The setting that decides which actions Claude can take without asking you. |
| **Plan mode** | A permission mode in which Claude researches and proposes a plan but makes no edits until you approve. |
| **Plugin** | An installable bundle of skills, subagents, hooks, and MCP servers. |
| **Prompt** | The request you type to Claude. Also, the terminal's "ready for input" marker. |
| **Prompt injection** | Hidden instructions in a web page, file, or tool result that try to trick an AI into doing something you didn't ask for. |
| **Sandbox** | An isolated environment that limits which files and network addresses a command can reach. |
| **Skill** | A saved set of instructions, in a `SKILL.md` file, that Claude can use automatically or that you can run as a `/command`. |
| **Subagent** | A specialist helper with its own instructions and tools that Claude can delegate work to. |
| **Terminal** | A text window for typing commands to your computer. Also called a shell, console, or command prompt. |
| **Token** | A small chunk of text, roughly three-quarters of a word, used to measure AI usage. |
| **Worktree** | A separate working copy of a Git repository, so parallel tasks don't step on each other. |